

**INSTITUTO TECNOLÓGICO Y DE ESTUDIOS SUPERIORES DE
MONTERREY**



**Tecnológico
de Monterrey**

Analysis and Design of Advanced Algorithms

Integrated Activity 1

DIEGO ISRAEL DOMINGUEZ NAJERA (A01782831)

Introduction

This problem situation describes a scenario where information sent between two devices travels as a stream of bits, carrying a header, the actual data, and a tail. A malicious actor can intercept this stream, alter parts of it, and forward it to the receiver with its own code embedded inside, which can later be used to take control of the receiving device.

This activity asks us to build a program that supports three tasks related to that scenario: detecting whether known malicious code sequences appear inside a transmission and where, identifying a distinctive pattern (a palindrome) that could mark the start of the real data inside the transmission, and comparing two transmissions to find the longest sequence they share, which could point to a common piece of inserted code. The three algorithms discussed below are the ones we used to address those three tasks.

1. Pattern Search

Problem Description

The first part of the activity asks the program to check whether the malicious code files (mcode1.txt, mcode2.txt, mcode3.txt) appear as an exact sequence of characters inside each transmission file (transmission1.txt, transmission2.txt), and, when they do, we have to report the position where the match starts. This has to be done six times, three malicious codes against two transmissions. Because the transmission files can be large, whatever method we use to search cannot afford to re-read characters it has already processed.

Algorithm

Comparing the pattern against the text at every position, one by one, is wasteful: whenever a comparison fails partway through, this approach restarts from scratch at the next position, discarding all the partial matching already done. This is when KMP appears, KMP avoids that by reusing what a mismatch already tells us: which characters of the pattern matched right before it. If part of the pattern's beginning also reappears at the end of what was just matched, the pattern can slide forward directly to that point instead of restarting.

To know how far to slide the window, KMP first builds an LPS (longest prefix suffix) array: for each position in the pattern, the length of the longest prefix that is also a suffix ending there. It depends only on the pattern, so it is built once, before the text is touched. The search then uses two pointers, one on the text and one on the pattern. When characters match, both advance, but when they don't, the text pointer never moves backward. The LPS array moves the pattern pointer to the correct position instead, and the comparison continues from there. When the pattern pointer reaches the end of the pattern, a match is found and its starting position is reported.

Time and Space complexity

Building the LPS array takes $O(m)$ time: the inner loop can reassign the length variable, but it only decreases as many times as the index advances, so the total work is bounded by $2m$.

Searching the text takes $O(n)$ time by the same argument: the text pointer never moves backward, and the pattern pointer falls back only as many times as it previously advanced.

Total time complexity: $O(n + m)$, well below the $O(n \text{ times } m)$ worst case of the naive approach.

Space complexity: $O(m)$, for the LPS array, the only extra memory used.

Since the transmission files (n) are large and the malicious code files (m) are small, linear time in n with memory proportional to m keeps the program fast as transmissions grow.

2. Longest Palindromic Substring

Problem Description

The second part asks the program to find the longest mirrored sequence, a palindrome, inside each transmission file, and report where it starts and ends (counting from 1). The comparison happens at the character level, over the characters themselves, not over their binary representation.

Algorithm

The first idea might be expanding around every position and growing outward while characters match works, but it's $O(n^2)$, since nothing gets reused between positions, which isn't good enough for large files. Manacher's algorithm solves this in $O(n)$ by exploiting the fact that palindromes are symmetric: once we know one, we already know something about the positions mirrored around its center, so we don't have to compare from scratch.

We first transform the string by inserting a separator between every character and at both ends, so every palindrome ends up with odd length, avoiding separate cases for even and odd. Then, as we move through the string, we track the palindrome that reaches furthest to the right so far. For each position, if it falls inside that known palindrome, its mirrored position already tells us a length we can reuse, unless it would stretch past what we actually know, in which case we only trust it up to there; if it falls outside, we start from zero. Either way, we try to expand a bit further by comparing characters directly, and update our reference if this one reaches further right, keeping track of the best length found and where it occurred.

Since this all happens on the transformed string, the last step maps the result back to the original one and reports it in the 1-based numbering the activity asks for.

Time and Space complexity

Building the transformed string takes $O(n)$ time, since it's a single pass over the original string.

The main loop looks like it could be $O(n^2)$ because of the while loop nested inside the for loop, but the inner loop doesn't run freely: every successful expansion pushes right one position further, and right never moves backward, so across the whole execution it can only succeed n times total, not n times per position. This makes the whole main loop $O(n)$.

Total time complexity: $O(n)$, a clear improvement over the $O(n^2)$ expand-around-center approach, and what makes it practical for large transmission files.

Space complexity: $O(n)$, for the transformed string and the array p , both proportional to the input size.

3. Longest Common Substring

Problem Description

The last part asks the program to measure how similar the two transmission files are, by finding the longest sequence of characters that appears, contiguously, in both files, and reporting where it starts and ends in the first file. This is a substring problem, not a subsequence problem: the matching characters have to be consecutive, no gaps allowed.

Algorithm

We solved this with dynamic programming, building a table where each cell answers a specific question: if the match ends exactly at this position in both files, how long is it? We shift the indexing by one so positions count from 1, matching the output format the activity asks for.

Filling each cell follows two rules: if the characters match, the value grows by one from whatever match already existed one position earlier in both files; if they don't, the run breaks completely and resets to zero. That reset is what separates this from longest common subsequence, where a mismatch would carry the previous value forward instead of discarding it, since subsequences allow gaps and substrings don't.

Each cell only depends on the row right above it, so we never need the full table, just the current row and the previous one, swapping after each pass. As we fill each row, we track the largest value seen and where it occurred, which gives us the ending position in the first file, and the starting position follows directly from that. If no match is ever found, we report both positions as zero instead of computing a meaningless range.

Time and Space complexity

Filling the table takes $O(n \text{ times } m)$ time: the outer loop runs n times, the inner one m times, and each cell does constant work, one comparison, one assignment, one check against the running maximum.

Space complexity: $O(m)$, since only two arrays of size $m+1$ are kept, the current row and the previous one, instead of the full table.

This matters for the project because a full n by m table would need one integer per pair of positions, which grows fast for large transmission files; keeping only two rows keeps memory linear instead of quadratic.

Conclusion

For me, working on Manacher's was the hardest part of all the activity, mostly because getting the mirror logic right is a little abstract. It's easy to understand the symmetry idea but easy to mess up the edge cases between when to trust the mirror and when to expand from scratch. Debugging it against "ABACABA" by hand helped a lot to catch that.

Seeing the other two parts come together, KMP reusing partial matches and the substring DP resetting on mismatches, made it clear how much a small insight, reusing what you already know instead of starting over, drives all three solutions. And I think that's really what this activity was about: picking the right algorithm isn't just about getting a correct answer, it's about not wasting work you've already done.